

CMSC424: Database Design

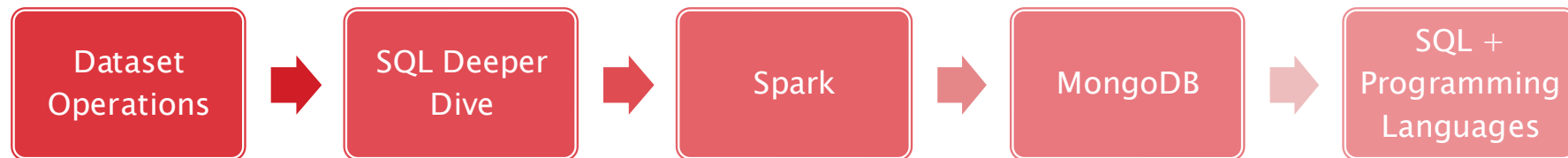
Module: Query Languages

Instructor: Amol Deshpande
amol@umd.edu

Learning Goals

- ▶ Key abstraction: A set or multi-set of objects or key-value pairs or tuples, called a “dataset”
- ▶ Basic operations on datasets
 - Take one or more datasets as input, produce a dataset as output
- ▶ Different ways to specify the above operations in different systems
 - Relational Algebra, SQL, MongoDB, Apache Spark (basics)
- ▶ Advanced SQL features
 - Focus on what’s possible, less on syntax
- ▶ Homework 1 (& Quiz 1) on the first three topics, Programming Assignment 1 covers last topic

Plan



Common operations on datasets: filter, map, join, outer join, aggregate, etc. Some common syntax patterns across systems

How to compose complex SQL queries
Transactions in SQL

Operating on RDDs
Dataframes and SQL interface

Operating on JSONs
Differences from SQL

Different ways to combine SQL and Programming Languages

Things we won't cover in detail in class

- SQL: Data update (CRUD) statements (e.g., insert/delete/update tuples, create tables, etc)
- SQL: Views, Triggers, Functions/procedures, Window Functions, Ranking, Recursion (Programming assignment 1)
- SQL: NULLs and Unknown (self-study – will provide materials)
- More complex operations in Spark or MongoDB (e.g., updates in MongoDB)
- SQL: Transactions and ACID properties (covered later)

CMSC424: Database Design

Module: Query Languages

Operations on Datasets

Instructor: Amol Deshpande
amol@cs.umd.edu

Inputs and Outputs: Datasets

- ▶ Sets or “multi-sets” or “collections” of ..
 - Objects $\rightarrow$ key-value pairs $\rightarrow$ tuples

o1
o2
o3
o4

Set of objects (e.g., images, JSON documents)

k1	o1
k2	o2
k3	o3
k4	o4

Set of key-value pairs (e.g., pairs of (userid, profile))

o1
o1
o1
o4

Multi-set of objects

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

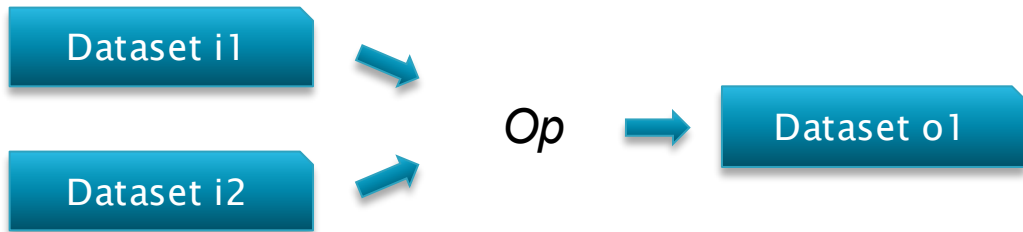
*Set of tuples
aka Relation/Table*

Operations on Datasets

Unary Operator



Binary Operator



- ▶ An operation involving “objects” typically requires some explicit code (e.g., in python) to interpret it
 - i.e., the “schema”/“structure” is implicit in the code
- ▶ An operation that works on a collection of “objects” also works on a collection of “key-value pairs” or “tuples”
- ▶ Operations with more than 2 inputs possible, but less common

1. Select/Find/Match/Filter

- ▶ Input: Object Collection, Output: Object Collection
 - So also works on Relations
- ▶ Select only those objects that match the condition
 - Need a predicate or a “function” that returns a boolean

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10



Select tuples where
 $A = B$ and $D > 5$



A	B	C	D
α	α	1	7
β	β	23	10

1. Select/Find/Match/Filter

- ▶ Input: Object Collection, Output: Object Collection
 - So also works on Relations
- ▶ Select only those objects that match the condition
 - Need a predicate or a “function” that returns a Boolean

Input

o1	
o2	
o3	
o4	



Select images with
cats



o1	
o2	
o4	

Need a function of the form:

`boolean has_a_cat(image_data)`

1. Select/Find/Match/Filter: Language Constructs?

- ▶ Relational Algebra (only on relations)
 - Symbol: σ (sigma)
- ▶ SQL
 - Uses “where” clause

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$$\sigma_{A=B \wedge D > 5}(r)$$

A	B	C	D
α	α	1	7
β	β	23	10

select *
from r
where r.A = r.B and r.D > 5

*“r.” optional if no
ambiguity*

1. Select/Find/Match/Filter: Language Constructs?

- ▶ Apache Spark (and many other similar frameworks)
 - Use “filter” operation
 - No explicit schema though, so need to provide code

Input I

o1	
o2	
o3	
o4	



`I.filter(lambda x: has_cat(x))`



Output O

o1	
o2	
o4	

Interpretation:

```
for each object x in I:
    if has_cat(x):
        O.append(x)
```

1. Select/Find/Match/Filter: Language Constructs?

- ▶ Apache Spark (and many other similar frameworks)
 - Use “filter” operation
 - No explicit schema though, so need to provide code

Input I: key-value pairs

k1	o1	
k2	o2	
k3	o3	
k4	o4	



`I.filter(lambda x: has_cat(x[1]))`



Output O

k1	o1	
k2	o2	
k4	o4	

Interpretation:

```
for each object x in I:
    if has_cat(x[1]):
        O.append(x)
```

2. Project/Map

- ▶ Input: Object Collection, Output: Object Collection
- ▶ For each object, output a transformed object

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10



For each input tuple, output
a tuple with:
column C and
square of column D



C	Dsq
1	49
5	49
12	9
23	100

2. Project/Map

- ▶ Input: Object Collection, Output: Object Collection
- ▶ For each object, output a transformed object

Input: a collection of log records

Output: IP addresses

```
199.72.81.55 - -  
[01/Jul/1995:00:00:01 -  
0400] "GET /history/apollo/  
HTTP/1.0" 200 6245
```

```
unicomp6.unicomp.net - -  
[01/Jul/1995:00:00:06 -  
0400] "GET  
/shuttle/countdown/  
HTTP/1.0" 200 3985
```

...

...

map: use regex to
extract IP address

```
199.72.81.55
```

```
unicomp6.uni  
comp.net
```

....

...

2. Project/Map

- ▶ Relational Algebra (only on relations)
 - Symbol: π (pi)
- ▶ SQL
 - Uses “select” clause

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\pi_{C, D * D}(r)$

C	Dsq
1	49
5	49
12	9
23	100

select C, D * D as Dsq
from r

2. Project/Map

- ▶ Apache Spark (and many other similar frameworks)
 - Use “map” operation
 - No explicit schema though, so need to provide code

Input: a collection of log records

Output: IP addresses

```
199.72.81.55 --  
[01/Jul/1995:00:00:01 -  
0400] "GET /history/apollo/  
HTTP/1.0" 200 6245
```

```
unicomp6.unicomp.net --  
[01/Jul/1995:00:00:06 -  
0400] "GET  
/shuttle/countdown/  
HTTP/1.0" 200 3985
```

...

...

```
I.map(lambda line:  
    re.match(r'^(\S+)', line)  
    .group(1))
```

Interpretation:

```
for each object x in I:  
    O.append(re.match(...).group(1))
```

```
199.72.81.55
```

```
unicomp6.uni  
comp.net
```

....

...

2. Project/Map

- ▶ Apache Spark (and many other similar frameworks)
 - Use “map” operation
 - No explicit schema though, so need to provide code

log_table(s)

Output: IP addresses

```
199.72.81.55 --  
[01/Jul/1995:00:00:01 -  
0400] "GET /history/apollo/  
HTTP/1.0" 200 6245
```

```
unicomp6.unicomp.net --  
[01/Jul/1995:00:00:06 -  
0400] "GET  
/shuttle/countdown/  
HTTP/1.0" 200 3985
```

...

...

```
l.map(lambda line:  
    re.match(r'^(\S+)', line)  
    .group(1))
```

```
199.72.81.55
```

```
unicomp6.uni  
comp.net
```

```
....
```

```
...
```

```
SELECT regexp_extract(s, '^(\S+)', 1) as host  
FROM log_table
```

2. Project/Map

- ▶ Relational Algebra: No duplicates in a relation
- ▶ SQL: allows duplicates by default

Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

$\pi_{A,D}(r)$

A	D
α	7
α	7
β	3
β	10

A	D
α	7
β	3
β	10

select A, D
from r

select distinct A, D
from r

SQL Deeper Dive: Basic Query Structure

select $A_1, A_2, \dots, A_n$ ← Attributes or expressions
from $r_1, r_2, \dots, r_m$ ← Relations (or queries returning tables)
where P ← Predicates

Find the names of all instructors:

```
select name  
from instructor
```

<i>name</i>
Srinivasan
Wu
Mozart
Einstein
El Said
Gold
Katz
Califieri
Singh
Crick
Brandt
Kim

Figure 3.2 Result of “**select** *name* **from** *instructor*”.

SQL Deeper Dive: Basic Query Structure

select $A_1, A_2, \dots, A_n$ ← Attributes or expressions
from $r_1, r_2, \dots, r_m$ ← Relations (or queries returning tables)
where P ← Predicates

Find the names of all instructor departments:

```
select dept_name  
from instructor
```

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

Figure 3.3 Result of “`select dept_name from instructor`”.

SQL Deeper Dive: Basic Query Structure

select $A_1, A_2, \dots, A_n$ ← Attributes or expressions
from $r_1, r_2, \dots, r_m$ ← Relations (or queries returning tables)
where P ← Predicates

Remove duplicates:
select distinct *name*
from *instructor*

Find the names of all instructors:
select *name*
from *instructor*

Order the output:
select distinct *name*
from *instructor*
order by *name* **asc**

Apply some filters (predicates):
select *name*
from *instructor*
where *salary* > 80000 **and** *dept_name* = 'Finance';

SQL Deeper Dive: Basic Query Constructs

Select all attributes:

```
select *  
from instructor
```

Expressions in the select clause:

```
select name, salary < 100000  
from instructor
```

Find the names of all instructors:

```
select name  
from instructor
```

More complex filters:

```
select name  
from instructor  
where (dept_name != 'Finance' and salary > 75000)  
or (dept_name = 'Finance' and salary > 85000);
```

A filter with a subquery:

```
select name  
from instructor  
where dept_name in (select dept_name from  
department where budget < 100000);
```

SQL Deeper Dive: Basic Query Constructs

Renaming tables or output column names:

```
select i.name, i.salary * 2 as double_salary  
from instructor i  
where i.salary < 80000 and i.name like '%g_';
```

Find the names of all instructors:

```
select name  
from instructor
```

More complex expressions:

```
select concat(name, concat(', ', dept_name))  
from instructor;
```

Careful with NULLs:

```
select name  
from instructor  
where salary < 100000 or salary >= 100000;
```

Wouldn't return the instructor with NULL salary (if any)

3. mapValues

- ▶ Input: Collection of (key, value) pairs; Output: Collection of (key, value) pairs
- ▶ Only apply the “map” operation to the value
- ▶ Available in Spark, straightforward to do in SQL

Input t

k1	o1
k1	o2
k2	o3
k3	o4

t.mapValues(f)

k1	f(o1)
k1	f(o2)
k2	f(o3)
k3	f(o4)

Assuming schema: (key, value)
select key, f(value)
from r

4. flatMap/unwind

- ▶ Input: Object collection, Output: Object collection
- ▶ For each row, apply a function that may generate ≥ 0 objects

Input I

o1
o2
o3
o4

$f(o1) = [o1', o2', o3']$

$f(o2) = []$

$f(o3) = [o4']$

$f(o4) = [o5']$

I.flatMap(f)

o1'
o2'
o3'
o4'
o5'

4. flatMap: Example

Input: a collection of sentences

"ACT I"
"SCENE I. Before LEONATO'S house."
"But few of any sort, and none of name."
...

`flatMap(lambda x:
x.split())`

Output: a collection of words

"ACT"
"I"
"SCENE"
"I."
"Before"
"LEONATO'S"
"house"
....

No real equivalent in SQL or Relational Algebra
Can achieve this through "unnest" or "lateral join"

5. Group(By)

- ▶ Input: Collection of (k, v) pairs, Output: Collection of (k, [v]) pairs
 - k = key, v = value
- ▶ Group the input rows by the “key”
- ▶ For relations, need to specify which attribute to use as the “key” (group by attribute)
- ▶ Relational Algebra: No support (can't have sets as values)
- ▶ Other Names: “nest” has somewhat similar behavior but not identical

Input t

k1	o1
k1	o2
k2	o3
k3	o4

t.groupByKey()

k1	[o1, o2]
k2	[o3]
k3	[o4]

Assuming schema: (key, value)

```
select key, array_agg(value) as value_arr
from r
group by key
```

5. GroupBy Example

Input: a collection
of user ratings

```
user1 product1 5.0
```

```
user1 product2 1.0
```

```
user1 product3 5.0
```

```
user2 product26 5.0
```

map(f), where:

def f (s):

 splits = s.split(" ")

 return (splits[0],
 (splits[1], splits[2]))

```
("user1", ("product1", 5.0))
```

```
("user1", ("product2", 1.0))
```

```
("user1", ("product3", 5.0))
```

```
("user2", ("product26", 5.0))
```

```
....
```

groupByKey



```
("user1", [("product1", 5.0), ("product2", 1.0),  
          ("product3", 5.0)])
```

```
("user2", [("product26", 5.0)])
```

```
....
```

5. GroupBy Example

```
[university=# select dept_name, array_agg(name) from instructor group by dept_name;
```

dept_name	array_agg
Finance	{Wu,Singh}
History	{"El Said",Califieri}
Physics	{Einstein,Gold}
Music	{Mozart}
Comp. Sci.	{Srinivasan,Katz,Brandt}
Biology	{Crick}
Elec. Eng.	{Kim}

(7 rows)

```
university=# █
```

6. Group(By) Aggregates

- ▶ Input: Collection of (k, v) pairs, Output: Collection of (k, v) pairs
- ▶ Also called “reduceByKey” or “aggregateByKey”
 - Need to specify a “reduce” or “aggregate” function
- ▶ Group values by key, and apply a provided “function” to get a single value
- ▶ SQL has a predefined set of functions (SUM, COUNT, MAX, ...)
- ▶ Relational Algebra: Not in basic set of operations

Input t

k1	o1
k1	o2
k2	o3
k3	o4

t.reduceByKey(f)

k1	f(o1, o2)
k2	f(o3)
k3	f(o4)

Typically f is a binary function, applied repeatedly

6. Group(By) Aggregates in SQL

Split the tuples into groups, and computer the aggregate for each group

```
select dept_name, avg (salary)
from instructor
group by dept_name;
```

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

6. Group(By) Aggregates in SQL

- ▶ SQL has a predefined set of functions (SUM, COUNT, MAX, ...)

```
[university=# select dept_name, count(*)
```

```
[university=# from student
```

```
[university=# group by dept_name;
```

dept_name	count
-----------	-------

Finance	1
---------	---

History	1
---------	---

Physics	3
---------	---

Music	1
-------	---

Comp. Sci.	4
------------	---

Biology	1
---------	---

Elec. Eng.	2
------------	---

(7 rows)

```
[university=# select dept_name, max(id)
```

```
from student
```

```
group by dept_name;
```

dept_name	max
-----------	-----

Finance	23121
---------	-------

History	19991
---------	-------

Physics	70557
---------	-------

Music	55739
-------	-------

Comp. Sci.	76543
------------	-------

Biology	98988
---------	-------

Elec. Eng.	98765
------------	-------

(7 rows)

6. Group(By) Aggregates in SQL

Find the average salary of instructors in the Computer Science

```
select avg(salary)  
from instructor  
where dept_name = 'Comp. Sci';
```

Other common aggregates:

max, min, sum, count, stdev, ...

```
select count (distinct ID)  
from teaches  
where semester = ' Spring' and year = 2010
```



Aggregate result can be used as a scalar.

Find instructors with max salary:

```
select *  
from instructor  
where salary = (select max(salary) from instructor);
```

7. Unnest

- ▶ Opposite of “nest” (group by)
- ▶ Similar to “flatMap” and “unwind” (in MongoDB)
 - But defined for relational algebra (extended to handle sets as values)
- ▶ Useful abstraction to deal with array data (e.g., inside of JSONs)

Table t

k1	[o1, o2]
k2	[o3]
k3	[o4]

unnest

k1	o1
k1	o2
k2	o3
k3	o4

8. Set Union/Intersection/Difference

- ▶ Input: Two collections, Output: One collection
- ▶ SQL support: union/except/intersection
 - Requires collections (tables) to have the same schema
 - Removes duplicates by default (most SQL operations don't remove duplicates)
 - Can use "union all" etc., to preserve duplicates
- ▶ Relational algebra: $\cup$ (union), $\cap$ (intersection), $-$ (difference)

Relation r, s

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$:

A	B
α	1
α	2
β	1
β	3

$r - s$:

A	B
α	1
β	1

8. Set Union/Intersection/Difference

Find courses that ran in Fall 2009 or Spring 2010

```
(select course_id from section where semester = 'Fall' and year = 2009)  
union  
(select course_id from section where semester = 'Spring' and year = 2010);
```

In both:

```
(select course_id from section where semester = 'Fall' and year = 2009)  
intersect  
(select course_id from section where semester = 'Spring' and year = 2010);
```

In Fall 2009, but not in Spring 2010:

```
(select course_id from section where semester = 'Fall' and year = 2009)  
except  
(select course_id from section where semester = 'Spring' and year = 2010);
```

8. Set Union/Intersection/Difference: Duplicates

Union/Intersection/Except eliminate duplicates in the answer (the other SQL commands don't) (e.g., try 'select dept_name from instructor').

Can use "union all" to retain duplicates.

NOTE: The duplicates are retained in a systematic fashion (for all SQL operations)

Suppose a tuple occurs m times in r and n times in s , then, it occurs:

- | $m + n$ times in r **union all** s
- | $\min(m, n)$ times in r **intersect all** s
- | $\max(0, m - n)$ times in r **except all** s

9. Cartesian Product

- ▶ Input: Two collections
- ▶ Output: A single collection that pairs every object from one collection with every object from the other

Relation r, s

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

- ▶ Note: a “set” product will result in nested output
 - First row would be: $((\alpha, 1), (\alpha, 10, a))$
 - Relation algebra flattens it by definition

9. Cartesian Product

- ▶ SQL does Cartesian Product for all tables in the FROM clause
- ▶ Can also use CROSS JOIN
 - `select * from R cross join S`
- ▶ Spark also has a “cartesian” operations
 - `r.cartesian(s)`
- ▶ Doesn't seem particularly useful??

```
university=# select * from R;
```

a	b
1	x
2	y

(2 rows)

```
university=# select * from S;
```

c	d
10	p
20	q
30	r

(3 rows)

```
university=# select * from R, S;
```

a	b	c	d
1	x	10	p
1	x	20	q
1	x	30	r
2	y	10	p
2	y	20	q
2	y	30	r

(6 rows)

9. Cartesian Product

Cartesian product:
select *
from *instructor*, *teaches*

for i in instructor:
 for t in teaches:
 print(i + t)

inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Physics	95000	10101	CS-101	1	Fall	2009
15151	Mozart	Physics	95000	10101	CS-315	1	Spring	2010
15151	Mozart	Physics	95000	10101	CS-347	1	Fall	2009
15151	Mozart	Physics	95000	10101	FIN-201	1	Spring	2010
15151	Mozart	Physics	95000	15151	MU-199	1	Spring	2010
15151	Mozart	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	10101	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Figure 3.6 The Cartesian product of the *instructor* relation with the *teaches* relation.

10. “Join” Operators

- ▶ Input: Two collections
- ▶ Output: A single collection that pairs every object from one collection with every object from the other, but only those that satisfy some condition

Relation r, s

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

$$r \bowtie_{A=C} s$$

Equivalent to:

$$\sigma_{A=C}(r \times s)$$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

10. “Join” Operators

- ▶ Many variations – some really more like “selection” operators

Cartesian product:

```
select *  
from R, S
```

A	B
a	1
b	1
c	2

R

×

B	C
1	x
3	y
4	z

S

=

R.A	R.B	S.B	S.C
a	1	1	x
a	1	3	y
a	1	4	z
b	1	1	x
b	1	3	y
b	1	4	z
c	2	1	x
c	2	3	y
c	2	4	z

10. “Join” Operators: Natural Join

Natural Join:

```
select *  
from R natural join S
```

A	B
a	1
b	1
c	2

R



B	C
1	x
3	y
4	z

S



R.A	B	S.C
a	1	x
b	1	x

Equivalent to:

```
select R.A, R.B, S.C  
from R, S  
where R.B = S.B
```

Equivalent to:

```
select R.A, R.B, S.C  
from R join S on (R.B = S.B)
```

Equivalent to:

```
select R.A, R.B, S.C  
from R join S on (B)
```

10. “Join” Operators: Natural Join

```
select *  
from instructor natural join teaches
```

```
for i in instructor:  
    for t in teaches:  
        if i.ID == t.ID:  
            print(i + t)
```

With one ID column removed

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Figure 3.8 The natural join of the *instructor* relation with the *teaches* relation.

10. “Join” Operators with >2 Tables

3-Table Query to get a list of instructor-teaches-course information:

```
select i.name as instructor_name, c.title as course_name  
from instructor i, course c, teaches  
where i.ID = teaches.ID and c.course_id = teaches.course_id;
```

Beware of unintended common names (happens often)

You may think the following query has the same result as above – it doesn't

```
select name, title  
from instructor natural join course natural join teaches;
```

I prefer avoiding “natural joins” for that reason

Note: On the small dataset, the above two have the same answer, but not on the large dataset. Large dataset has cases where an instructor teaches a course from a different department.

10. “Join” Operators: Missing Tuples in Output

Natural Join:

```
select *  
from R natural join S
```

A	B
a	1
b	1
c	2

R



B	C
1	x
3	y
4	z

S



R.A	B	S.C
a	1	x
b	1	x

Often need the “non-matching” tuples in the result

10. “Join” Operators: Left Outerjoin

```
select *  
from R natural left outer join S
```

A	B
a	1
b	1
c	2

R



B	C
1	x
3	y
4	z

S

=

R.A	B	S.C
a	1	x
b	1	x
c	2	NULL

NULL is a keyword in SQL

```
select *  
from R left outer join S on (R.B = S.B)
```

=

```
select *  
from R natural join S on (R.B = S.B)  
UNION ALL  
select R.A, R.B, NULL  
from R where R.B NOT IN  
  (select S.B from S)
```

10. “Join” Operators: Right Outerjoin

```
select *  
from R right natural outer join S
```

A	B		B	C		R.A	B	S.C
a	1		1	x		a	1	x
b	1		3	y		b	1	x
c	2		4	z		NULL	3	y
						NULL	4	z
R			S					

```
select *  
from R right outer join S on (R.B = S.B)
```

```
select *  
from R natural join S on (R.B = S.B)  
UNION ALL  
select NULL, S.B, S.C  
from S where S.B NOT IN  
  (select R.B from R)
```

10. “Join” Operators: Full Outerjoin

```
select *  
from R natural full outer join S
```

A	B
a	1
b	1
c	2

R



B	C
1	x
3	y
4	z

S



R.A	B	S.C
a	1	x
b	1	x
c	2	NULL
NULL	3	y
NULL	4	z

```
select *  
from R full outer join S on (R.B = S.B)
```

10. “Join” Operators: Spark??

`join(otherDataset, [numPartitions])`

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through `leftOuterJoin`, `rightOuterJoin`, and `fullOuterJoin`.

Input I1

k1	o1
k1	o2
k2	o3
k3	o4

Input I2

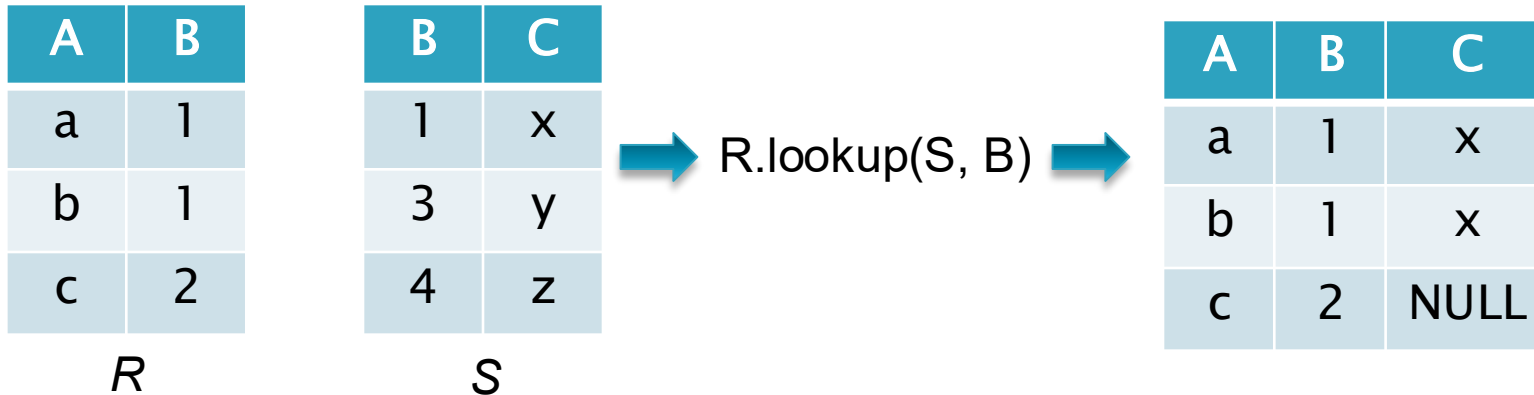
k1	w1
k1	w2
k2	w3
k2	w4

I1.join(I2)

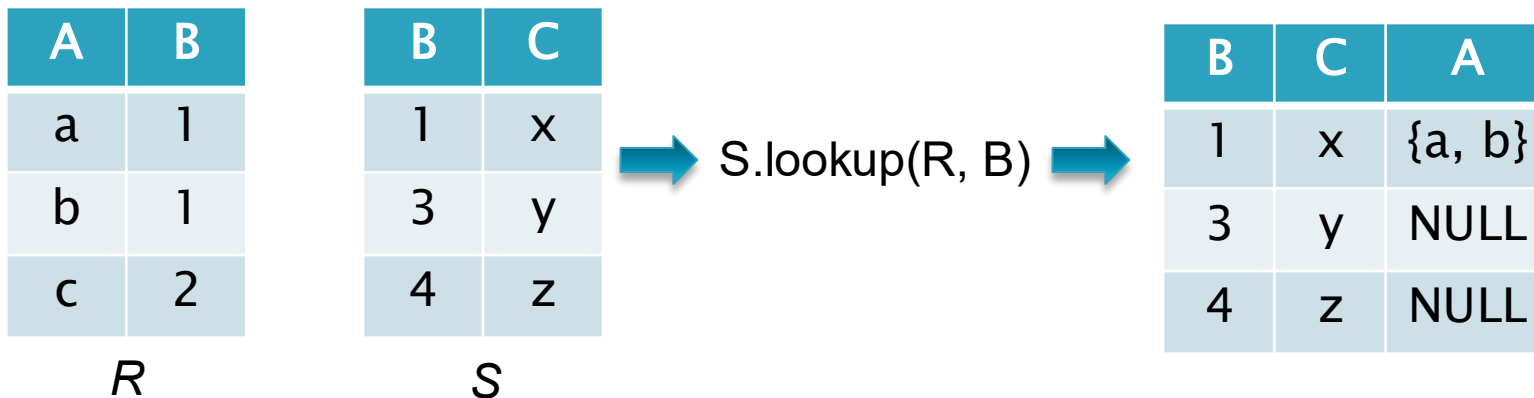
k1	(o1, w1)
k1	(o1, w2)
k1	(o2, w1)
k1	(o2, w2)
k2	(o3, w3)
k2	(o3, w4)

10. “Join” Operators: Lookup

An asymmetric version of (left outer) join, found in **Excel**, MongoDB, etc.



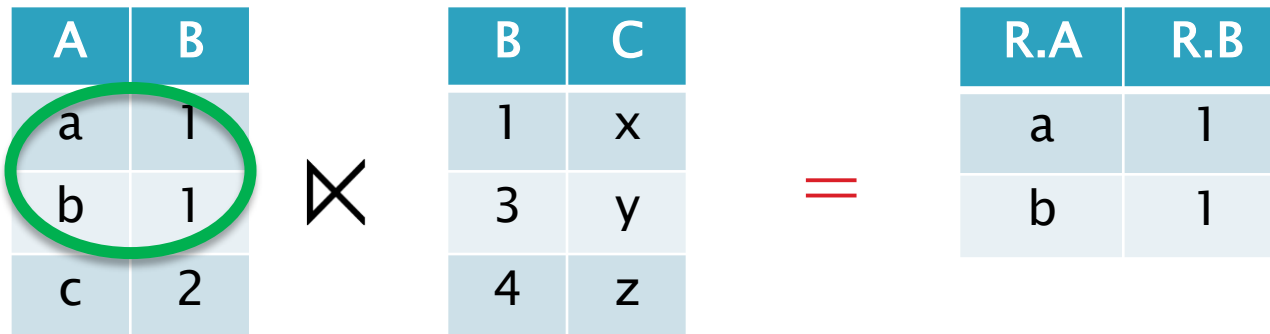
Need a way to deal with multiple matches in the right input



11. Semi-joins

R SEMI-JOIN S = tuples of R that do have a “match” in S

Not an SQL keyword, but useful concept to understand – often implemented in database systems as an operator



Can be written in SQL as:

```
select *  
from R  
where B in (select B from S);
```

11. Semi-joins

R SEMI-JOIN S = tuples of R that do have a “match” in S

Not an SQL keyword, but useful concept to understand – often implemented in database systems as an operator

A	B
a	1
b	1
c	2

 $\bowtie$

B	C
1	x
3	y
4	z

 $=$

R.A	R.B
a	1
b	1

A	B
a	1
b	1
c	2

 $\bowtie$

B	C
1	x
3	y
4	z

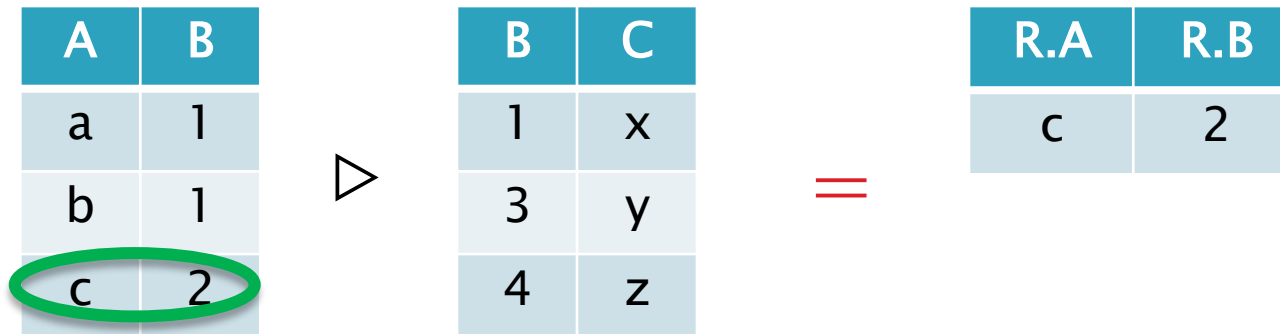
 $=$

S.B	S.C
1	x

12. Anti-joins

$R \text{ ANTI-JOIN } S$ = tuples of R that do NOT have a “match” in S

Not an SQL keyword, but useful concept to understand – often implemented in database systems as an operator



Can be written in SQL as:

```
select *  
from R  
where B not in (select B from S);
```

12. Anti-joins

$R \text{ ANTI-JOIN } S$ = tuples of R that do NOT have a “match” in S

Not an SQL keyword, but useful concept to understand – often implemented in database systems as an operator

A	B
a	1
b	1
c	2

 $\triangleright$

B	C
1	x
3	y
4	z

 $=$

R.A	R.B
c	2

A	B
a	1
b	1
c	2

 $\triangleleft$

B	C
1	x
3	y
4	z

 $=$

S.B	S.C
3	y
4	z

Joins: Summary

- ▶ Tables: $r(A, B)$, $s(B, C)$

name	Symbol	SQL Equivalent	RA expression
cross product	$\times$	select * from r, s;	$r \times s$
natural join	$\bowtie$	natural join	$\pi_{r.A, r.B, s.C} \sigma_{r.B = s.B}(r \times s)$
theta join	$\bowtie_{\theta}$	from .. where θ ;	$\sigma_{\theta}(r \times s)$
equi-join	$\bowtie_{\theta}$ (<i>theta must be equality</i>)		
left outer join	$r \bowtie\!\!\!\bowtie s$	left outer join (with “on”)	(see previous slide)
full outer join	$r \bowtie\!\!\!\bowtie\!\!\!\bowtie s$	full outer join (with “on”)	–
(left) semijoin	$r \bowtie\!\!\!\bowtie s$	none	$\pi_{r.A, r.B}(r \bowtie\!\!\!\bowtie s)$
(left) antijoin	$r \bowtie\!\!\!\bowtie\!\!\!\bowtie s$	none	$r - \pi_{r.A, r.B}(r \bowtie\!\!\!\bowtie s)$

Key Takeaways

- ▶ Databases typically present the data as one or more “tables” or collections
 - One exception: property graph databases (e.g., Neo4j)
 - Even there, many similarities
 - ▶ Most common query languages support the same set of core operations
 - Select/filter, project/map, flatMap, group by, reduce/aggregate
 - Joins of various kind to combine information across multiple tables/collections
 - Set union/intersection
 - ▶ Parity develops across popular languages by necessity
- 

Key Takeaways

- ▶ How multiple operations are “combined” has some key differences

<i>student</i>
<u>ID</u>
name
dept_name
tot_cred

<i>advisor</i>
<u>s_id</u>
<u>i_id</u>

<i>instructor</i>
<u>ID</u>
name
dept_name
salary

Find advisor/advisee names where student department is “CS”.

```
select student.name, instructor.name
from student, advisor, instructor
where student.id = s_id and i_id = instructor.id
and student.dept_name = “CS”
```

$\Pi_{\text{student.name, instructor.name}}(\sigma_{\text{student.id=s_id} \wedge \text{i_id} = \text{instructor.id} \wedge \text{student.dept_name} = \text{“CS”}}(\text{student} \times \text{advisor} \times \text{instructor}))$

```
student.filter(lambda s: s.dept.name = “CS”).join(advisor).join(instructor)
    .map(lambda x: (x.student_name, x.instructor_name))
```

Note: Syntax not fully accurate – need some more notation

CMSC424: Database Design

Module: Query Languages

Deeper Dive into SQL

Instructor: Amol Deshpande
amol@cs.umd.edu

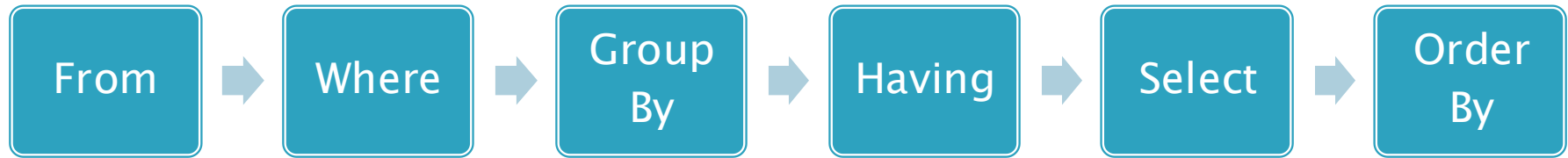
History

- ▶ IBM Sequel language developed as part of System R project at the IBM San Jose Research Laboratory
- ▶ Renamed Structured Query Language (SQL)
- ▶ ANSI and ISO standard SQL:
 - SQL-86, SQL-89, SQL-92
 - SQL:1999, SQL:2003, SQL:2008, SQL:2023
- ▶ Commercial systems offer most, if not all, ANSI SQL features, plus varying feature sets from later standards and special proprietary features.
 - Not all examples here may work on your particular system.
- ▶ Several alternative syntaxes to write the same queries

Different Types of Constructs

- ▶ **Data definition language (DDL):** Defining/modifying schemas
 - **Integrity constraints:** Specifying conditions the data must satisfy
 - **View definition:** Defining views over data
 - **Authorization:** Who can access what
- ▶ **Data-manipulation language (DML):** Insert/delete/update tuples, queries
- ▶ **Transaction control:**
- ▶ **Embedded SQL:** Calling SQL from within programming languages
- ▶ **Creating indexes, Query Optimization control...**

SQL Basic Evaluation Order



```
select course_id, semester, year, sec_id, avg (tot_cred)  
from student, takes  
where student.ID= takes.ID and year = 2017  
group by course_id, semester, year, sec_id  
having count (ID) >= 2;  
order by course_id;
```

Subqueries can be used anywhere
Allow for “composing” more complex queries

One of the reasons why the pipe syntax is easier to understand

Composition Pattern 1

- ▶ **Pattern 1:** Construct scalars or sets that can be used in other queries
- ▶ Scalars often used for comparisons or for simplifying “selects”
- ▶ e.g., compute the max salary across instructor and use it in a comparison

```
SELECT *  
FROM instructor  
WHERE salary = (  
    SELECT MAX(salary)  
    FROM instructor  
);
```

Composition Pattern 1

- ▶ **Pattern 1:** Construct scalars or sets that can be used in other queries
- ▶ Check if a tuple attribute belongs to a set
 - Can also done through a join, but requires care

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2009
      AND course_id IN (
        SELECT course_id
        FROM section
        WHERE semester = 'Spring' AND year = 2010
      );
```

- ▶ Can also use “exists” to check whether the set is empty

```
WHERE EXISTS (subquery)
```

Composition Pattern 2

- ▶ **Pattern 2:** Construct a table that can be used in further queries
- ▶ In the “from” clause
 - The subquery creates a table: (dept_name, avg_salary), from which we then select

```
SELECT dept_name, avg_salary
FROM (
    SELECT dept_name, AVG(salary) AS avg_salary
    FROM instructor
    GROUP BY dept_name
) AS T
WHERE avg_salary > 42000;
```

Can also do this using HAVING

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

- ▶ Using “WITH” – Common Table Expressions (CTEs)
 - Preferred

```
WITH dept_avg AS (
    SELECT dept_name, AVG(salary) AS avg_salary
    FROM instructor
    GROUP BY dept_name
)
SELECT dept_name
FROM dept_avg
WHERE avg_salary > 42000;
```

Composition Pattern 2

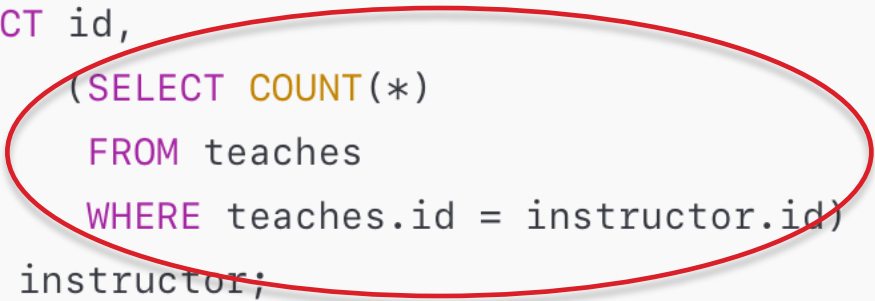
- ▶ **Pattern 2:** Construct a table that can be used in further queries
- ▶ Can do chains of these through WITH

```
with dept_total (dept_name, value) as  
    (select dept_name, sum(salary)  
     from instructor  
     group by dept_name),  
dept_total_avg(value) as  
    (select avg(value)  
     from dept_total)  
select dept_name  
from dept_total, dept_total_avg  
where dept_total.value > dept_total_avg.value;
```

Composition Pattern 3

- ▶ **Pattern 3:** Construct a “function” for computing something that requires other relations
- ▶ These are also called “correlated” subqueries
- ▶ Often used in “select” and “where” clauses

```
SELECT id,  
  (SELECT COUNT(*)  
   FROM teaches  
   WHERE teaches.id = instructor.id)  
FROM instructor;
```



Think of this like
a function that
takes in a `instructor.id`
as input

- ▶ Clearer to read, but query optimizers often “rewrite” internally

Composition Pattern 3

- ▶ **Pattern 3:** Construct a “function” for computing something that requires other relations
- ▶ These are also called “correlated” subqueries
- ▶ Often used in “select” and “where” clauses
- ▶ A function to count the number of instructors in a department

```
select dept_name,  
       (select count(*)  
        from instructor  
        where department.dept_name = instructor.dept_name)  
       as num_instructors  
from department;
```

(1) Find classrooms over capacity, i.e., with more students than capacity.

(2) Find students who haven't taken the pre-requisite for a course they did take

(3) Instructors who taught 4 courses in a single day in a single semester, along with the day and the semester



(4) Create a pivot table, with rows being course_ids, and columns being Grades (let's assume "grade" takes values A, B, C, D, F)

So we want something like:

	A	B	C	D	F	
CMSC424	120	90	100	20		20

(5) For each course, find the most lenient instructor, i.e., the instructor who appears to have the highest average grade

(6) Find pairs of students with identical courses taken throughout -- output should be a table with two studentid columns

Transactions

- ▶ A transaction is a sequence of queries and update statements executed as a single unit
 - Transactions are started implicitly and terminated by one of
 - **commit work**: makes all updates of the transaction permanent in the database
 - **rollback work**: undoes all updates performed by the transaction.
- ▶ In most database systems, each SQL statement that executes successfully is automatically committed.
 - Each transaction would then consist of only a single statement
 - Automatic commit can usually be turned off, allowing multi-statement transactions, but how to do so depends on the database system
 - Another option in SQL:1999: enclose statements within
 - begin atomic**
 - ...
 - end**

Examples of Transactions

insert into students values (...)

update instructor
set salary = salary * 1.03

enrolled = select count(*)
 from takes
 where (course_info) = (CMSC 424, 201, Fall 2022)
if enrolled < capacity for the room:
 insert new student into takes for that course

(Modify prerequisites)
delete (CMSC422, CMSC351)
insert (CMSC422, CMSC320)

(Add a new section for a course for a given room and instructor)
if no section currently in that room:
 insert a tuple into “sections” with that room
 insert a tuple into “teaches”

(Remove a section)
delete from takes for that section
delete from teaches for that section
delete tuple from section

(Switch the advisor for a student)
delete old tuple with that s_id
add new tuple with that s_id and new advisor

Examples of Transactions

What if only part of the transaction was completed?

update instructor
set salary = salary * 1.03

(Modify prerequisites)
delete (CMSC422, CMSC351)
insert (CMSC422, CMSC320)

(Remove a section)
delete from takes for that section
delete from teaches for that section
delete tuple from section

(Switch the advisor for a student)
delete old tuple with that s_id
add new tuple with that s_id and new advisor

Examples of Transactions

```
enrolled = select count(*)  
            from takes  
            where (course_info) = (CMSC 424, 201, Fall 2022)  
if enrolled < capacity for the room:  
    insert student A into takes for that course
```

```
enrolled = select count(*)  
            from takes  
            where (course_info) = (CMSC 424, 201, Fall 2022)  
if enrolled < capacity for the room:  
    insert student B into takes for that course
```

What if two different students tried to enroll at the same time?

Transactions and ACID

- ▶ Transaction: A sequence of database actions enclosed within special tags
- ▶ Properties:
 - **Atomicity**: Entire transaction or nothing
 - **Consistency**: Transaction, executed completely, takes database from one consistent state to another
 - **Isolation**: Concurrent transactions appear to run in isolation
 - **Durability**: Effects of committed transactions are not lost
- ▶ Consistency: Transaction programmer needs to guarantee that
 - DBMS can do a few things, e.g., enforce constraints on the data
- ▶ Rest: DBMS guarantees

How does..

- ▶ .. this relate to *queries* that we discussed ?
 - Queries don't update data, so durability and consistency not relevant
 - Would want concurrency
 - Consider a query computing total balance at the end of the day
 - Would want isolation
 - What if somebody makes a *transfer* while we are computing the balance
 - Typically not guaranteed for such long-running queries

Views: Virtual Expressions as Tables

- ▶ A view consisting of courses and sections for Physics in Fall 2009

```
create view physics_fall_2009 as  
  select course.course_id, sec_id, building, room_number  
  from course, section  
  where course.course_id = section.course_id  
        and course.dept_name = 'Physics'  
        and section.semester = 'Fall'  
        and section.year = '2009';
```

Find all physics fall 2009 courses in a building.

```
select course_id  
from physics_fall_2009  
where building = 'Watson';
```

SQL Functions

- ▶ Function to count number of instructors in a department

```
create function dept_count (dept_name varchar(20))  
  returns integer  
  begin  
    declare d_count integer;  
    select count ( * ) into d_count  
    from instructor  
    where instructor.dept_name = dept_name  
    return d_count;  
end
```

- ▶ Can use in queries

```
select dept_name, budget  
from department  
where dept_count (dept_name ) > 12
```

SQL Procedures

- ▶ Same function as a procedure

```
create procedure dept_count_proc (in dept_name varchar(20),  
                                out d_count integer)
```

```
begin
```

```
    select count(*) into d_count
```

```
    from instructor
```

```
    where instructor.dept_name = dept_count_proc.dept_name
```

```
end
```

- ▶ But use differently:

```
    declare d_count integer;
```

```
    call dept_count_proc( 'Physics' , d_count);
```

- ▶ **HOWEVER: Syntax can be wildly different across different systems**
 - Was put in place by DBMS systems before standardization
 - Hard to change once customers are already using it

More features...

- ▶ Ranking
 - Can assign “ranks” to tuples based on some attribute values
- ▶ Window functions
- ▶ Recursion
 - SQL supports recursion through “with recursive”
 - Makes SQL Turing complete, although implementations like PostgreSQL pretty limited
- ▶ Arrays and Unnest
 - `CREATE TABLE R(name text, schedule text[][]);`
 - Quite useful in practice, but must keep array sizes low
- ▶ Pivots, Rollups, Cubes
 - Features useful for aggregating and visualizing large datasets
- ▶ Authorization/Security
 - Control access per table, and sometimes per column or per row basis

CMSC424: Database Design

Module: Query Languages

Apache Spark

Instructor: Amol Deshpande
amol@umd.edu

Apache Spark

❓ Book Chapters

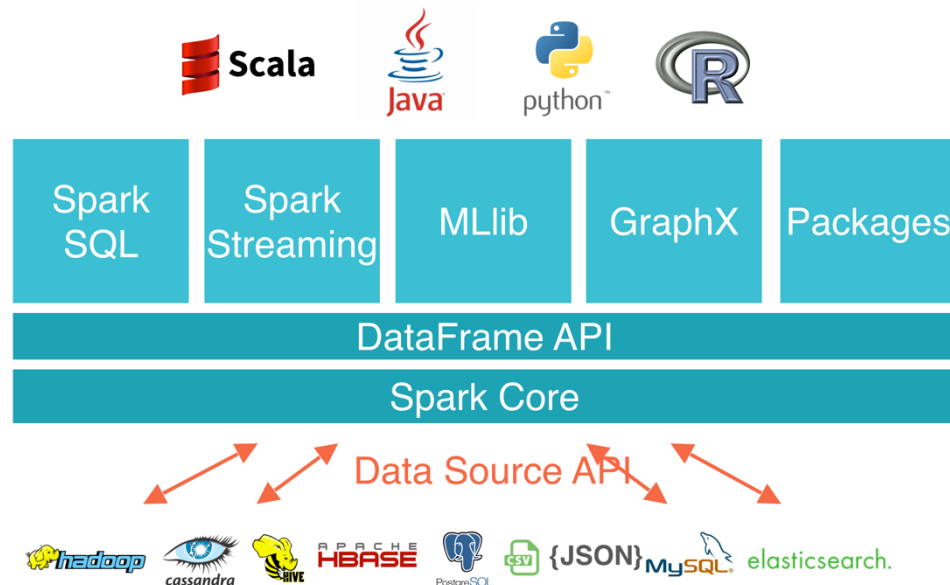
- ★ 10.4 (7TH EDITION) covers this topic, but Spark programming guide is a better resource
- ★ Assignments will refer to the programming guide

❓ Key topics:

- ★ A Resilient Distributed Dataset (RDD)
- ★ Operations on RDDs

Spark

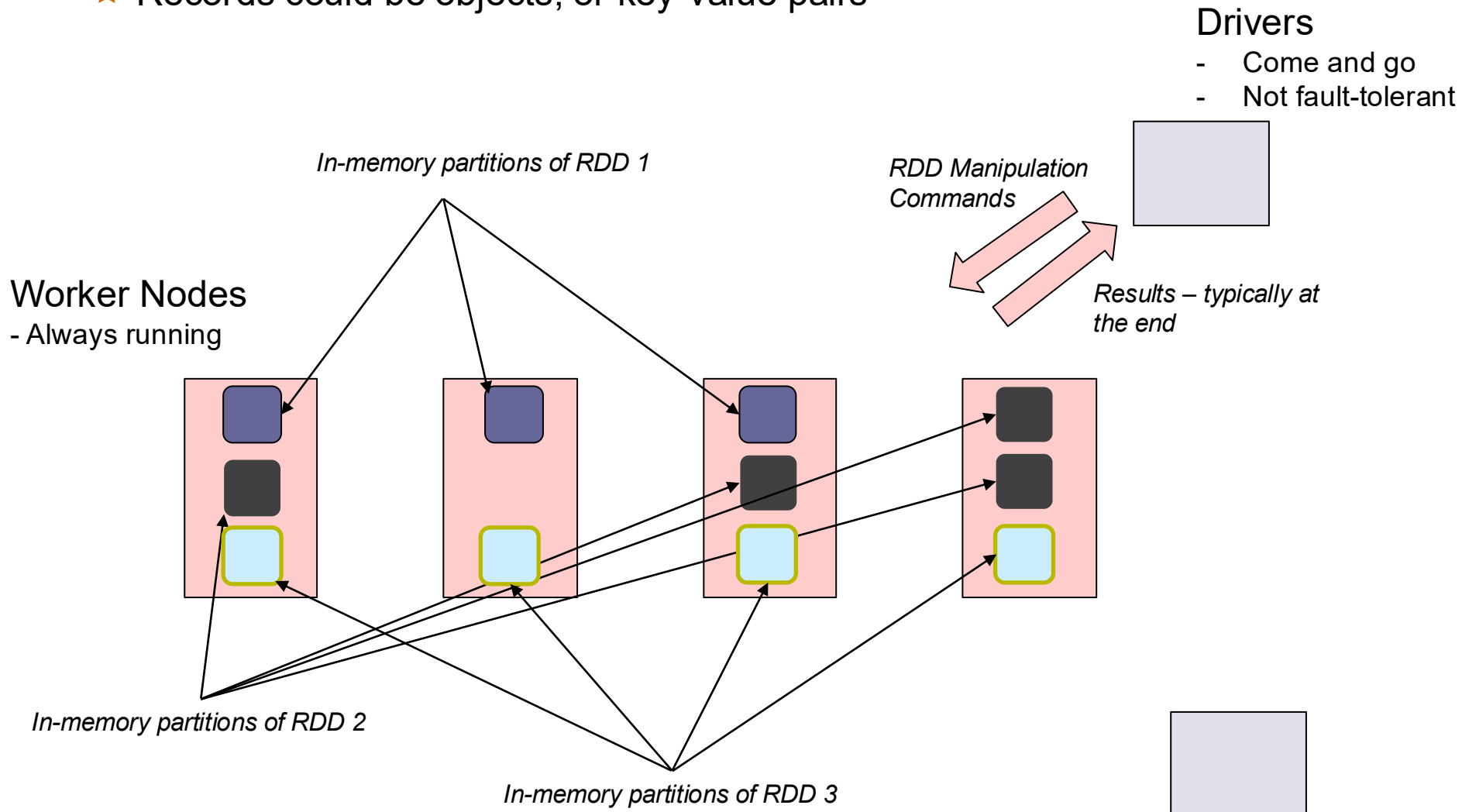
- n Open-source, distributed cluster computing framework
- n Much better performance than predecessors (e.g., Hadoop MapReduce) through in-memory caching and pipelining
- n Originally provided a low-level RDD-centric API, but today, most of the use is through the “Dataframes” (i.e., relations) API
 - ★ Dataframes support relational operations like Joins, Aggregates, etc.



Resilient Distributed Dataset (RDD)

RDD = Collection of records stored across multiple machines in-memory

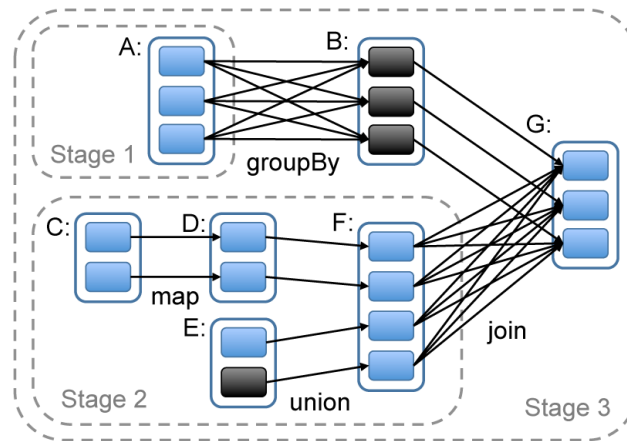
★ Records could be objects, or key-value pairs



Spark

n Why “Resilient”?

- ★ Can survive the failure of a worker node
- ★ Spark maintains a “lineage graph” of how each RDD partition was created
- ★ If a worker node fails, the partitions are recreated from its inputs
- ★ Only a small set of well-defined operations are permitted on the RDDs
 - But the operations usually take in arbitrary “map” and “reduce” functions



n Fault tolerance for the “driver” is trickier

- ★ Drivers have arbitrary logic (cf., the programs you are writing)
- ★ In some cases (e.g., Spark Streaming), you can do fault tolerance
- ★ But in general, driver failure requires a restart

Example Spark Program

Initialize RDD by reading the textFile and partitioning

If textFile stored on HDFS, it is already partitioned – just read each partition as a separate RDD partition

Split each line into words, creating an RDD of words

For each word, output (word, 1), creating a new RDD

Do a group-by SUM aggregate to count the number of times each word appears

Driver

```
from pyspark import SparkContext

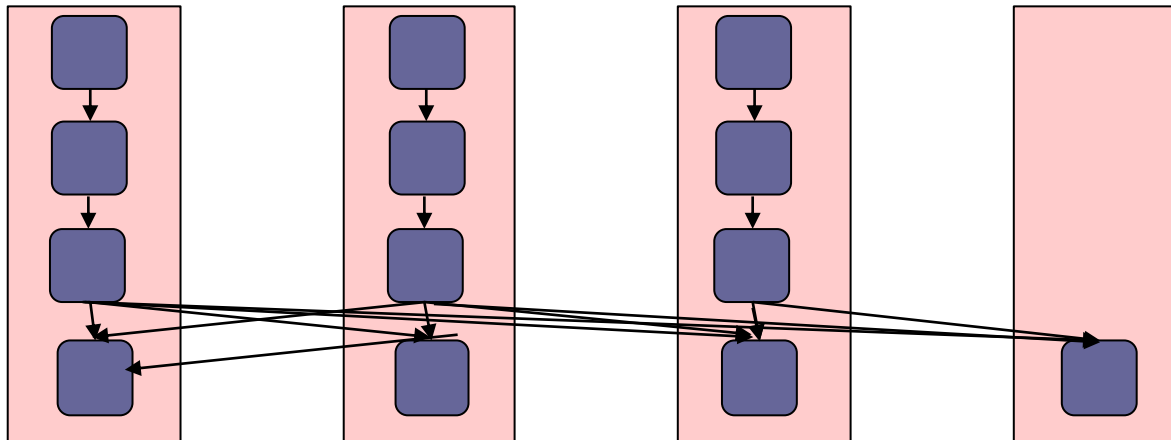
sc = SparkContext("local", "Simple App")

textFile = sc.textFile("README.md")

counts = textFile
    .flatMap(lambda line: line.split(" "))
    .map(lambda word: (word, 1))
    .reduceByKey(lambda a, b: a + b)

print(counts.take(100))
```

Retrieve 100 of the values in the final RDD



Spark

- n Operations often take in a "function" as input
- n Using the inline "lambda" functionality

```
flatMap(lambda line: line.split(" "))
```

- n Or a more explicit function declaration

```
def split(line):  
    return line.split(" ")  
  
flatMap(split)
```

- n Similarly "reduce" functions essentially tell Spark how to do pairwise aggregation

```
reduceByKey(lambda a, b: a + b)
```

- ★ Spark will apply this to the dataset pair of values at a time
- ★ Difficult to do something like "median"

Spark: Map

$x_1, x_2, \dots$ can be anything,
including documents,
images, text files, tuples,
dicts, etc.

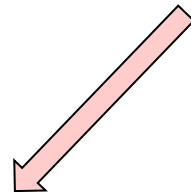
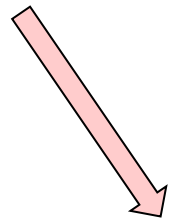
InputRDD: $[x_1, x_2, \dots, x_n]$



`map(lambda x: x + 1)`

`def fn(x):
 return x+1`

`map(fn)`



outputRDD: $[x_1+1, x_2+1, \dots, x_n+1]$

InputRDD: $[x_1, x_2, \dots, x_n]$



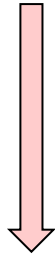
`map(fn)`



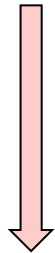
outputRDD: $[fn(x_1), fn(x_2), \dots, fn(x_n)]$

Spark: flatMap

InputRDD: [(a1, b1), (a2, b2), ...]



flatMap(lambda x: [x[0], x[1]])



outputRDD: [a1, b1, a2, b2, ...]

InputRDD: ['the little brown fox...', ...]



flatMap(lambda x: x.split())

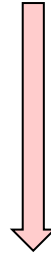


outputRDD: ['the', 'little', 'brown', ...]

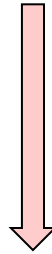
Spark: groupByKey

InputRDD: [(a1, b1), (a2, b2), (a1, b3), (a1, b4), (a2, b5)...]

InputRDD must be a collection of 2-tuples
Usually called (Key, Value) pairs
Value can be anything (e.g., dicts, tuples, bytes)



groupByKey()

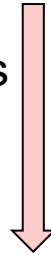


outputRDD: [(a1, [b1, b3, b4, ...]), (a2, [b3, b5,...]), ...]

Spark: reduceByKey

InputRDD: [(a1, b1), (a2, b2), (a1, b3), (a1, b4), (a2, b5)...]

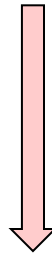
InputRDD must be a collection of 2-tuples
Usually called (Key, Value) pairs



reduceByKey(func)

```
def func(V1, V2):  
    return V3
```

All of V1, V2, and V3
be of the same type



outputRDD: [(a1, ...func(func(b1, b3), b4)...),
(a2, ...func(func(b2, b5), ...)...),]

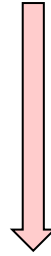
"func" executed in parallel in a pairwise fashion

Spark: join

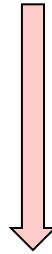
InputRDD1: [(a1, b1), (a2, b2), (a1, b3), (a1, b4), (a2, b5)...]

InputRDD2: [(a1, c1), (a2, c2), (a1, c3), (a1, c4), (a2, c5)...]

InputRDD1 and InputRDD2 both must
be a collection of 2-tuples



`inputRDD1.join(inputRDD2)`



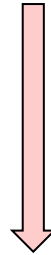
outputRDD: [(a1, (b1, c1)),
 (a1, (b1, c3)),
 (a1, (b1, c4)),
 ]

Spark: cogroup

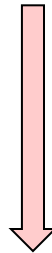
InputRDD1: [(a1, b1), (a2, b2), (a1, b3), (a1, b4), (a2, b5)...]

InputRDD2: [(a1, c1), (a2, c2), (a1, c3), (a1, c4), (a2, c5)...]

InputRDD1 and InputRDD2 both must
be a collection of 2-tuples



`inputRDD1.cogroup(inputRDD2)`



outputRDD: [(a1, ([b1, b3, b4, ...], [c1, c3, c4, ...])),
 (a2, ([b2, b5, ...], [c2, c5, ...])), ...
]

RDD Operations

Transformations

The following table lists some of the common transformations supported by Spark. Refer to the RDD API doc ([Scala](#), [Java](#), [Python](#), [R](#)) and pair RDD functions doc ([Scala](#), [Java](#)) for details.

Transformation	Meaning
map (<i>func</i>)	Return a new distributed dataset formed by passing each element of the source through a function <i>func</i> .
filter (<i>func</i>)	Return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true.
flatMap (<i>func</i>)	Similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item).
mapPartitions (<i>func</i>)	Similar to map, but runs separately on each partition (block) of the RDD, so <i>func</i> must be of type <code>Iterator<T> => Iterator<U></code> when running on an RDD of type T.
mapPartitionsWithIndex (<i>func</i>)	Similar to mapPartitions, but also provides <i>func</i> with an integer value representing the index of the partition, so <i>func</i> must be of type <code>(Int, Iterator<T>) => Iterator<U></code> when running on an RDD of type T.
sample (<i>withReplacement</i> , <i>fraction</i> , <i>seed</i>)	Sample a fraction <i>fraction</i> of the data, with or without replacement, using a given random number generator seed.
union (<i>otherDataset</i>)	Return a new dataset that contains the union of the elements in the source dataset and the argument.
intersection (<i>otherDataset</i>)	Return a new RDD that contains the intersection of elements in the source dataset and the argument.
distinct (<i>numPartitions</i>)	Return a new dataset that contains the distinct elements of the source dataset.
groupByKey (<i>numPartitions</i>)	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. Note: If you are grouping in order to perform an aggregation (such as a sum or average) over each key, using <code>reduceByKey</code> or <code>aggregateByKey</code> will yield much better performance. Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional <code>numPartitions</code> argument to set a different number of tasks.
reduceByKey (<i>func</i> , <i>numPartitions</i>)	When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function <i>func</i> , which must be of type <code>(V, V) => V</code> . Like in <code>groupByKey</code> , the number of reduce tasks is configurable through an optional second argument.
aggregateByKey (<i>zeroValue</i>)(<i>seqOp</i> , <i>combOp</i> , <i>numPartitions</i>)	When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in <code>groupByKey</code> , the number of reduce tasks is configurable through an optional second argument.
sortByKey (<i>ascending</i> , <i>numPartitions</i>)	When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean

Actions

The following table lists some of the common actions supported by Spark. Refer to the RDD API doc ([Scala](#), [Java](#), [Python](#), [R](#)) and pair RDD functions doc ([Scala](#), [Java](#)) for details.

Action	Meaning
reduce (<i>func</i>)	Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect ()	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count ()	Return the number of elements in the dataset.
first ()	Return the first element of the dataset (similar to <code>take(1)</code>).
take (<i>n</i>)	Return an array with the first <i>n</i> elements of the dataset.
takeSample (<i>withReplacement</i> , <i>num</i> , <i>seed</i>)	Return an array with a random sample of <i>num</i> elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
takeOrdered (<i>n</i> , <i>ordering</i>)	Return the first <i>n</i> elements of the RDD using either their natural order or a custom comparator.
saveAsTextFile (<i>path</i>)	Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call <code>toString</code> on each element to convert it to a line of text in the file.
saveAsSequenceFile (<i>path</i>) (Java and Scala)	Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like Int, Double, String, etc).
saveAsObjectFile (<i>path</i>) (Java and Scala)	Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using <code>SparkContext.objectFile()</code> .
countByKey ()	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
foreach (<i>func</i>)	Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an Accumulator or interacting with external storage systems. Note: modifying variables other than Accumulators outside of the <code>foreach()</code> may result in undefined behavior. See Understanding closures for more details.

Dataframes Example

```
def basic_df_example(spark):
    # $example on:create_df$
    # spark is an existing SparkSession
    df = spark.read.json("examples/src/main/resources/people.json")
    # Displays the content of the DataFrame to stdout
    df.show()
    # +----+-----+
    # | age | name |
    # +----+-----+
    # | null | Michael |
    # | 30 | Andy |
    # | 19 | Justin |
    # +----+-----+
    # $example off:create_df$

    # $example on:untyped_ops$
    # spark, df are from the previous example
    # Print the schema in a tree format
    df.printSchema()
    # root
    # |-- age: long (nullable = true)
    # |-- name: string (nullable = true)

    # Select only the "name" column
    df.select("name").show()
    # +-----+
    # | name |
    # +-----+
    # | Michael |
    # | Andy |
    # | Justin |
    # +-----+

    # Select everybody, but increment the age by 1
    df.select(df['name'], df['age'] + 1).show()
    # +-----+-----+
    # | name | (age + 1) |
    # +-----+-----+
    # | Michael | null |
    # | Andy | 31 |
    # | Justin | 20 |
    # +-----+-----+
```

```
# Select people older than 21
df.filter(df['age'] > 21).show()
# +----+-----+
# | age | name |
# +----+-----+
# | 30 | Andy |
# +----+-----+
```

```
# Count people by age
df.groupBy("age").count().show()
# +----+-----+
# | age | count |
# +----+-----+
# | 19 | 1 |
# | null | 1 |
# | 30 | 1 |
# +----+-----+
# $example off:untyped_ops$
```

```
sqlDF = spark.sql("SELECT * FROM people")
sqlDF.show()
# +----+-----+
# | age | name |
# +----+-----+
# | null | Michael |
# | 30 | Andy |
# | 19 | Justin |
# +----+-----+
# $example off:run_sql$
```

```
# $example on:global_temp_view$
# Register the DataFrame as a global temporary view
df.createGlobalTempView("people")
```

```
# Global temporary view is tied to a system preserved database `global_temp`
spark.sql("SELECT * FROM global_temp.people").show()
# +----+-----+
# | age | name |
# +----+-----+
# | null | Michael |
# | 30 | Andy |
# | 19 | Justin |
# +----+-----+
```

Summary

- n Spark is a popular and widely used framework for large-scale computing
- n Simple programming interface
 - ★ You don't need to typically worry about the parallelization
 - ★ That's handled by Spark transparently
 - ★ In practice, may need to fiddle with number of partitions etc.
- n Managed services supported by several vendors including Databricks (started by the authors of Spark), Cloudera, etc.
- n Many other concepts that we did not discuss
 - ★ Shared accumulator and broadcast variables
 - ★ Support for Machine Learning, Graph Analytics, Streaming, and other use cases
- n Alternatives include: Apache Tez, Flink, and several others

CMSC424: Database Design

Module: Query Languages

MongoDB

Instructor: Amol Deshpande
amol@umd.edu

MongoDB: History 1



- ▶ A prototypical NoSQL database introduced in 2009
- ▶ We saw:
 - SQL → Declarative, relational, normalized
 - Spark → Operator pipelines over datasets
- ▶ In contrast, MongoDB:
 - Document-oriented
 - Schema-flexible (although most deployments have schemas)
 - Query language centered on collections
 - Aggregations pipelines (Spark-like)
- ▶ Think: JSON + Spark-style operators (with some SQL-like constructs thrown in)

MongoDB Data Model

MongoDB	Relational
Database	Database
Collection	Relation
Document	Row/Record
Field	Column

```
{
  item: "journal",
  instock: [
    { loc: "A", qty: 5 },
    { loc: "C", qty: 15 }
  ],
  tags: ["blank", "red"],
  dim: [14, 21]
}
```

Document = {..., field: value, ...}

Where value can be:

- Atomic
- A document
- An array of atomic values
- An array of documents

Can also mix and match, e.g., array of atomics and documents, or array of arrays

Questions:

- *Where would “instock” go in an RDBMS?*
- *Where would “tags” go?*

MongoDB Data Model

▶ MongoDB

- One-to-many relationships inside a document
- Natural storage of hierarchical data
- No normalization required
- No schema migration needed to add fields

▶ Vs Relational Systems

- One-to-many relationships often separated out, needing JOINS
- Schema evolution is trickier

▶ Tradeoff: Flexibility vs structure/constraints/performance

Retrieval Queries (find)

find() = SQL SELECT-WHERE

```
db.inventory.find(  
  { status: "D", qty: { $gte: 50 } },  
  { item: 1, _id: 0 }  
)
```

```
SELECT item  
FROM inventory  
WHERE status = 'D'  
AND qty >= 50;
```

Key differences

- Predicates expressed as JSON documents
- Projection expressed as JSON documents
- One collection at a time (similar to other pipe languages)

Retrieval Queries Template

`db.collection.find(<predicate>, optional <projection>)`

returns documents that **match <predicate>**

keep fields as specified in **<projection>**

both **<predicate>** and **<projection>** expressed as documents
in fact, most things are documents!

`db.inventory.find( { } )`

returns all documents

```
> db.inventory.find()
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d993"), "item" : "journal", "qty" : 25, "size" : { "h" : 14, "w" : 21, "uom" : "cm" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d994"), "item" : "notebook", "qty" : 50, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d995"), "item" : "paper", "qty" : 100, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d996"), "item" : "planner", "qty" : 75, "size" : { "h" : 22.85, "w" : 30, "uom" : "cm" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d997"), "item" : "postcard", "qty" : 45, "size" : { "h" : 10, "w" : 15.25, "uom" : "cm" }, "status" : "A" }
```

Retrieval Queries (find)

```
> db.inventory.find()
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d993"), "item" : "journal", "qty" : 25, "size" : { "h" : 14, "w" : 21, "uom" : "cm" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d994"), "item" : "notebook", "qty" : 50, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d995"), "item" : "paper", "qty" : 100, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d996"), "item" : "planner", "qty" : 75, "size" : { "h" : 22.85, "w" : 30, "uom" : "cm" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d997"), "item" : "postcard", "qty" : 45, "size" : { "h" : 10, "w" : 15.25, "uom" : "cm" }, "status" : "A" }
```

Binary operators

```
{ field: { $operator: value } }
```

e.g.,

```
{ qty: { $gt: 30 } }
```

AND: { status : "D", qty : { \$gte : 50 } }

OR: { \$or: [{ status : "D" }, { qty : { \$lt : 30 } }] }

Dot Notation (Nested Access)

```
{ "instock.qty": { $gte: 20 } }
```

```
{ "size.uom" : "cm", "size.h" : { $gt : 14 } }
```

Retrieval Queries (find)

```
> db.inventory.find()
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d993"), "item" : "journal", "qty" : 25, "size" : { "h" : 14, "w" : 21, "uom" : "cm" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d994"), "item" : "notebook", "qty" : 50, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "A" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d995"), "item" : "paper", "qty" : 100, "size" : { "h" : 8.5, "w" : 11, "uom" : "in" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d996"), "item" : "planner", "qty" : 75, "size" : { "h" : 22.85, "w" : 30, "uom" : "cm" }, "status" : "D" }
{ "_id" : ObjectId("5fb55fad75b4eb537dc3d997"), "item" : "postcard", "qty" : 45, "size" : { "h" : 10, "w" : 15.25, "uom" : "cm" }, "status" : "A" }
```

Arrays have more complex logic

- ▶ `find( { tags: ["red", "blank"] } )`
 - Exact match of array → notebook
- ▶ `find( { tags: "red" } )`
 - If one of the elements matches red → journal, notebook, paper, planner
- ▶ `find( { tags: "red", tags: "plain" } )`
 - If one matches red, one matches plain → paper

(Optional) Projection – can specify what fields to keep or omit

- ▶ `find({}, {item : 1, "instock.loc": 1, _id : 0})`
 - Keep item, keep only loc inside instock, omit _id
 - Can't mix and match (either specify keep or omit), except _id

Retrieval Queries: Summary

find() = SELECT <projection>
FROM Collection
WHERE <predicate>

limit() = LIMIT

sort() = ORDER BY

```
db.inventory.find(  
    { tags : red },  
    { _id : 0, instock : 0 } )  
.sort ( { "dim.0": -1, item: 1 } )  
.limit (2)
```

```
FROM  
WHERE  
SELECT  
ORDER BY  
LIMIT
```

Aggregation = Operator Pipelines

```
db.collection.aggregate([  
  { $match: {...} },  
  { $group: {...} },  
  { $sort: {...} }  
])
```

Compared to Spark

```
df.filter(...).groupBy(...).agg(...).sort(...)
```

*Note: Unfortunate use of
the word “aggregate”*

Example: State Population

```
db.zips.aggregate([
  { $group: { _id: "$state",
              totalPop: { $sum: "$pop" } } },
  { $match: { totalPop: { $gte: 15000000 } } },
  { $sort: { totalPop: -1 } }
])
```

\$group: _id specifies grouping attribute

\$match after \$group → HAVING

Explicit pipeline ordering

```
SELECT state, SUM(pop) AS totalPop
FROM zips
GROUP BY state
HAVING SUM(pop) >= 15000000
ORDER BY totalPop DESC;
```

Unwind: Flatten Arrays

```
{ item: "journal", tags: ["blank", "red"] }
```



```
{ item: "journal", tags: "blank" }  
{ item: "journal", tags: "red" }
```

```
{ $unwind: "$tags" }
```

Equivalent to:

- Spark flatMap
- SQL UNNEST

Unwinding Arrays

```
> db.inventory.find()
{ "_id" : ObjectId("5fb59ab9f50b800678c0e196"), "item" : "journal", "instock" : [ { "loc" : "A", "qty" : 5 }, { "loc" : "C", "qty" : 15 } ], "tags" : [ "blank", "red" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e197"), "item" : "notebook", "instock" : [ { "loc" : "C", "qty" : 5 } ], "tags" : [ "red", "blank" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e198"), "item" : "paper", "instock" : [ { "loc" : "A", "qty" : 60 }, { "loc" : "B", "qty" : 15 } ], "tags" : [ "red", "blank", "plain" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e199"), "item" : "planner", "instock" : [ { "loc" : "A", "qty" : 40 }, { "loc" : "B", "qty" : 5 } ], "tags" : [ "blank", "red" ], "dim" : [ 22.85, 30 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e19a"), "item" : "postcard", "instock" : [ { "loc" : "B", "qty" : 15 }, { "loc" : "C", "qty" : 35 } ], "tags" : [ "blue" ], "dim" : [ 10, 15.25 ] }
```

```
aggregate( [
  { $unwind : "$tags" },
  { $project : { _id : 0, instock: 0 } }
])
```

```
{ "item" : "journal", "tags" : "blank", "dim" : [ 14, 21 ] }
{ "item" : "journal", "tags" : "red", "dim" : [ 14, 21 ] }
{ "item" : "notebook", "tags" : "red", "dim" : [ 14, 21 ] }
{ "item" : "notebook", "tags" : "blank", "dim" : [ 14, 21 ] }
{ "item" : "paper", "tags" : "red", "dim" : [ 14, 21 ] }
{ "item" : "paper", "tags" : "blank", "dim" : [ 14, 21 ] }
{ "item" : "paper", "tags" : "plain", "dim" : [ 14, 21 ] }
{ "item" : "planner", "tags" : "blank", "dim" : [ 22.85, 30 ] }
{ "item" : "planner", "tags" : "red", "dim" : [ 22.85, 30 ] }
{ "item" : "postcard", "tags" : "blue", "dim" : [ 10, 15.25 ] }
```

Unwind: A Common Template

```
> db.inventory.find()
{ "_id" : ObjectId("5fb59ab9f50b800678c0e196"), "item" : "journal", "instock" : [ { "loc" : "A", "qty" : 5 }, { "loc" : "C", "qty" : 15 } ], "tags" : [ "blank", "red" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e197"), "item" : "notebook", "instock" : [ { "loc" : "C", "qty" : 5 } ], "tags" : [ "red", "blank" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e198"), "item" : "paper", "instock" : [ { "loc" : "A", "qty" : 60 }, { "loc" : "B", "qty" : 15 } ], "tags" : [ "red", "blank", "plain" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e199"), "item" : "planner", "instock" : [ { "loc" : "A", "qty" : 40 }, { "loc" : "B", "qty" : 5 } ], "tags" : [ "blank", "red" ], "dim" : [ 22.85, 30 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e19a"), "item" : "postcard", "instock" : [ { "loc" : "B", "qty" : 15 }, { "loc" : "C", "qty" : 35 } ], "tags" : [ "blue" ], "dim" : [ 10, 15.25 ] }
```

Q: Imagine if we want to find sum of qtys across items. How would we do this?

A common recipe in MQL queries is to *unwind* and then *group by*

```
aggregate( [
  { $unwind : "$instock" },
  { $group : {
    _id : "$item",
    totalqty : { $sum : "$instock.qty" }
  }}
])
```

```
{ "_id" : "notebook", "totalqty" : 5 }
{ "_id" : "postcard", "totalqty" : 50 }
{ "_id" : "journal", "totalqty" : 20 }
{ "_id" : "planner", "totalqty" : 45 }
{ "_id" : "paper", "totalqty" : 75 }
```

Joins in MongoDB (\$lookup)

```
{  
  $lookup: {  
    from: "otherCollection",  
    localField: "x",  
    foreignField: "y",  
    as: "results"  
  }  
}
```

Left outer equijoin
Results stored as an array
Often requires heavy projection cleanup

Joins in MongoDB (\$lookup)

```
> db.inventory.find()
{ "_id" : ObjectId("5fb59ab9f50b800678c0e196"), "item" : "journal", "instock" : [ { "loc" : "A", "qty" : 5 }, { "loc" : "C", "qty" : 15 } ], "tags" : [ "blank", "red" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e197"), "item" : "notebook", "instock" : [ { "loc" : "C", "qty" : 5 } ], "tags" : [ "red", "blank" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e198"), "item" : "paper", "instock" : [ { "loc" : "A", "qty" : 60 }, { "loc" : "B", "qty" : 15 } ], "tags" : [ "red", "blank", "plain" ], "dim" : [ 14, 21 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e199"), "item" : "planner", "instock" : [ { "loc" : "A", "qty" : 40 }, { "loc" : "B", "qty" : 5 } ], "tags" : [ "blank", "red" ], "dim" : [ 22.85, 30 ] }
{ "_id" : ObjectId("5fb59ab9f50b800678c0e19a"), "item" : "postcard", "instock" : [ { "loc" : "B", "qty" : 15 }, { "loc" : "C", "qty" : 35 } ], "tags" : [ "blue" ], "dim" : [ 10, 15.25 ] }
```

```
db.inventory.aggregate( [
  { $lookup : {from : "inventory", localField: "instock.loc", foreignField: "instock.loc", as:"otheritems"}},
  { $project : {_id : 0, tags : 0, dim : 0}}
])
```

```
{ "item" : "journal", "instock" : [ { "loc" : "A", "qty" : 5 }, { "loc" : "C", "qty" : 15 } ],
  "otheritems" : [
    { "_id" : ObjectId("5fb6f9605f0594e0227d3c24"), "item" : "journal",
      "instock" : [ { "loc" : "A", "qty" : 5 }, { "loc" : "C", "qty" : 15 } ],
      "tags" : [ "blank", "red" ], "dim" : [ 14, 21 ] },
    { "_id" : ObjectId("5fb6f9605f0594e0227d3c25"), .....}
    { "_id" : ObjectId("5fb6f9605f0594e0227d3c26"), .....},
    ...
  ]}
```

Updates

```
db.inventory.updateMany(  
  { "dim.0": { $lt: 15 } },  
  { $set: { status: "InvalidWidth" } }  
)
```

```
UPDATE inventory  
SET status = 'InvalidWidth'  
WHERE dim[0] < 15;
```

Unique aspects:

- Can update nested fields directly
- No schema definition required
- Document-level atomicity (i.e., some document may get updated, others may not)

Architectural Notes

- ▶ MongoDB is a distributed NoSQL database
 - ▶ Collections are partitioned/sharded based on a field [range-based]
 - Each partition stores a subset of documents
 - ▶ Each partition is replicated to help with failures
 - The replication is done asynchronously
 - Failures of the main partition that haven't been propagated will be lost
 - ▶ Limited heuristic-based query optimization
 - ▶ Atomic writes to documents within collections by default. Multi-document txns are discouraged (but now supported).
- 

SQL/Relational vs MongoDB vs Spark

Feature	SQL	MongoDB	Spark
Data model	Relational	JSON documents	DataFrames / RDDs
Schema	Fixed	Flexible	Fixed (schema-on-read)
Nesting	No (pure model)	Yes	Yes (structs)
Arrays	Not native	First-class	First-class
Joins	Core feature	\$lookup (awkward)	Core feature
Query style	Declarative	JSON-based + pipeline	Operator chaining
Optimization	Cost-based	Mostly heuristic	Cost-based (Catalyst)

CMSC424: Database Design

Module: Relational Model + SQL

SQL and Programming Languages

Instructor: Amol Deshpande
amol@umd.edu

SQL and Programming Languages

- ▶ Book Chapters (6th Edition)
 - Sections 5.1, 9.4.2
- ▶ Key Topics
 - Why use a programming language
 - Embedded SQL vs ODBC/JDBC
 - Object-relational impedance mismatch
 - Object-relational Mapping Frameworks

SQL and Programming Languages

- ▶ Programmers/developers more comfortable using a programming language like Java, Python, etc.
 - SQL not natural for many things
 - Performance issues in going back and forth to the database
- ▶ Need to deal with **impedance mismatch** between:
 - how data is represented in memory (typically as objects)
 - how it is stored (typically in a “normalized” relational schema)

Relational database (such as PostgreSQL or MySQL)

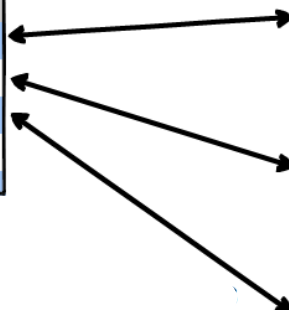
ID	FIRST_NAME	LAST_NAME	PHONE
1	John	Connor	+16105551234
2	Matt	Makai	+12025555689
3	Sarah	Smith	+19735554512
...	...	...	...

Python objects

```
class Person:  
    first_name = "John"  
    last_name = "Connor"  
    phone_number = "+16105551234"
```

```
class Person:  
    first_name = "Matt"  
    last_name = "Makai"  
    phone_number = "+12025555689"
```

```
class Person:  
    first_name = "Sarah"  
    last_name = "Smith"  
    phone_number = "+19735554512"
```



Option 1: JDBC/ODBC

- ▶ Use a standard protocol like JDBC (Java Database Connectivity) to talk to the database from the programming language

```
>>> import jaydebeapi
>>> conn = jaydebeapi.connect("org.hsqldb.jdbcDriver",
...                           "jdbc:hsqldb:mem:",
...                           ["SA", ""],
...                           "/path/to/hsqldb.jar",)
>>> curs = conn.cursor()
>>> curs.execute('create table CUSTOMER'
...               ' ("CUST_ID" INTEGER not null,'
...               '  "NAME" VARCHAR(50) not null,'
...               '  primary key ("CUST_ID"))'
...               )
>>> curs.execute("insert into CUSTOMER values (1, 'John')")
>>> curs.execute("select * from CUSTOMER")
>>> curs.fetchall()
[(1, u'John')]
>>> curs.close()
>>> conn.close()
```

- ▶ Doesn't solve impedance mismatch problem
 - Have to convert from the “result tuples” into “objects” and vice versa (when updating)

```

import java.sql.*;

public class JDBCExample
{

    public static void main(String[] argv) {
        System.out.println("----- PostgreSQL " + "JDBC Connection Testing -----");
        try {
            Class.forName("org.postgresql.Driver");
        } catch (ClassNotFoundException e) {
            System.out.println("Where is your PostgreSQL JDBC Driver? " + "Include in your library path!");
            e.printStackTrace();
            return;
        }

        System.out.println("PostgreSQL JDBC Driver Registered!");
        Connection connection = null;
        try {
            connection = DriverManager.getConnection("jdbc:postgresql://localhost:5432/olympics","vagrant", "vagrant");
        } catch (SQLException e) {
            System.out.println("Connection Failed! Check output console");
            e.printStackTrace();
            return;
        }

        if (connection != null) {
            System.out.println("You made it, take control your database now!");
        } else {
            System.out.println("Failed to make connection!");
            return;
        }

        Statement stmt = null;
        String query = "select * from players;";
        try {
            stmt = connection.createStatement();
            ResultSet rs = stmt.executeQuery(query);
            while (rs.next()) {
                String name = rs.getString("name");
                System.out.println(name + "\t");
            }
            stmt.close();
        } catch (SQLException e) {
            System.out.println(e);
        }
    }
}

```

Option 1: JDBC/ODBC

- ▶ WARNING: always use prepared statements when taking an input from the user and adding it to a query (**Related to the issue of SQL Injection attacks**)
 - NEVER create a query by concatenating strings
 - "insert into instructor values(' " + ID + " ', ' " + name + " ', " + " ' + dept name + " ', " + " ' balance + ')"
 - What if name is "D'Souza"?

```
PreparedStatement pstmt = conn.prepareStatement("insert into instructor values(?,?,?,?)");
pstmt.setString(1, "88877");
pstmt.setString(2, "Perry");
pstmt.setString(3, "Finance");
pstmt.setInt(4, 125000);
pstmt.executeUpdate();
pstmt.setString(1, "88878");
pstmt.executeUpdate();
```

- ▶ Python psycopg2 also has its own way of doing prepared statements

```
cur = conn.cursor()
for i, j in parameters:
    cur.execute( "select * from tables where i = %s and j = %s", (i, j))
    for record in cur: do_something_with(record)
```

Option 1: JDBC/ODBC

▶ JDBC Features

- Getting schemas, columns, primary keys
 - `DatabaseMetaData dbmd = conn.getMetaData()`
- Transaction control
 - `conn.commit()`, `conn.rollback()`
- Calling functions and procedures

▶ ODBC: Open Database Connectivity Standard

- Similar in many ways
- Older – designed by Microsoft and typically used in C, C++, like languages
 - Java supports as well but slower

Option 2: Embedded SQL

- ▶ SQL standard defines embeddings of SQL in a variety of programming languages such as C, C++, Java, Fortran, and PL/1
 - The language in which embedded is call “host” language

C++

```
int main() {
    EXEC SQL INCLUDE SQLCA;
    EXEC SQL BEGIN DECLARE SECTION;
        int OrderID;          /* Employee ID (from user)      */
        int CustID;           /* Retrieved customer ID       */
        char SalesPerson[10]  /* Retrieved salesperson name  */
        char Status[6]        /* Retrieved order status      */
    EXEC SQL END DECLARE SECTION;

    /* Set up error processing */
    EXEC SQL WHENEVER SQLERROR GOTO query_error;
    EXEC SQL WHENEVER NOT FOUND GOTO bad_number;

    /* Prompt the user for order number */
    printf("Enter order number: ");
    scanf_s("%d", &OrderID);

    /* Execute the SQL query */
    EXEC SQL SELECT CustID, SalesPerson, Status
        FROM Orders
        WHERE OrderID = :OrderID
        INTO :CustID, :SalesPerson, :Status;

    /* Display the results */
    printf("Customer number: %d\n", CustID);
    printf("Salesperson: %s\n", SalesPerson);
    printf("Status: %s\n", Status);
    exit();
}
```

Option 2: Embedded SQL

- ▶ SQL standard defines embeddings of SQL in a variety of programming languages such as C, C++, Java, Fortran, and PL/1
 - The language in which embedded is call “host” language
- ▶ Needs compiler support for the host language
 - The compiler needs to know what to do with the EXEC SQL commands
 - Hard to port
- ▶ Doesn't solve impedance mismatch problem
 - Have to convert from the “result tuples” into “objects” and vice versa (when updating)
- ▶ Not a preferred approach today

Option 3: Custom Libraries

- ▶ Often there are vendor-specific libraries that sometimes use internal protocols (and not JDBC/ODBC)
- ▶ e.g., python psycopg2 for PostgreSQL – although similar to JDBC calls, it uses the same proprietary protocol that 'psql' uses

```
conn = psycopg2.connect("dbname=olympics user=vagrant")
cur = conn.cursor()

totalscore = 0
for i in range(0, 14):
    # If a query is specified by -q option, only do that one
    if args.query is None or args.query == i:
        try:
            if interactive:
                os.system('clear')
            print("===== Executing Query {}".format(i))
            print(queries[i])
            cur.execute(queries[i])

            if i not in [5, 6, 8, 9]:
                ans = cur.fetchall()

                print("----- Your Query Answer -----")
                for t in ans:
                    print(t)
                print("")
```

Option 4: Object-relational Mappers

- ▶ Aimed at solving the impedance mismatch
 - Primarily for Web Application Development
- ▶ The ORM takes care of the mapping between objects and the database
 - Although largely designed around RDBMS, some ORMs support other databases as well
- ▶ The programmer works with objects, and never directly sees the SQL
 - Has pros (easier to use) and cons (performance and correctness issues)
- ▶ ORMs typically work with “Entities/Objects” and “Relationships”
 - Aligns well with the ER model that we will discuss next
 - We will cover Django constructs in more detail

Relational database (such as PostgreSQL or MySQL)

ID	FIRST_NAME	LAST_NAME	PHONE
1	John	Connor	+16105551234
2	Matt	Makai	+12025555689
3	Sarah	Smith	+19735554512
...	...	...	...

Python objects

```
class Person:  
    first_name = "John"  
    last_name = "Connor"  
    phone_number = "+16105551234"
```

```
class Person:  
    first_name = "Matt"  
    last_name = "Makai"  
    phone_number = "+12025555689"
```

```
class Person:  
    first_name = "Sarah"  
    last_name = "Smith"  
    phone_number = "+19735554512"
```

ORMs provide a bridge between
**relational database tables, relationships
and fields** and **Python objects**

Option 5: Other Mappers

- ▶ Many other “wrappers” on top of relational databases that offer different functionalities
 - In some cases, operations written in a higher-level language mapped to SQL
 - like what we saw for ORMs
 - Microsoft LINQ is also similar
 - Allows intermixing of code mapped to SQL and other code
 - In some cases, used to provide alternate data models to users
 - e.g., a thin layer that provides a graph data model, but stores data in a relational database
 - Most “RDF” databases built on top of SQL databases
- ▶ In today’s big data ecosystem, we see many many permutations how different tools (including databases) are combined together